# Vibecode 101

Build a useful app. Understand your decisions. Work with other people.

Teaching edition 1.0. Resources checked November 2, 2026. Dates below are relative until the organizer configures a cohort. This plan has not yet been validated through a classroom pilot.

A five-week, in-person course for adults with an idea and little or no programming experience. Direct an AI coding agent, build three small individual projects, and work in a team to finish one useful app. Understand and stand behind what you build, without memorizing programming syntax.

Ten class hours, plus about eight to ten hours of preparation and project work. Bring a laptop and internet access. The organizer must confirm access to one supported coding agent or a paired alternative before enrollment. Sponsored access is a plan, not a promise. No paid API or billing card is required for the core assignments.

Adults 18+ who can use a browser, manage basic files, and describe an app idea in English. No programming prerequisite. This version is designed for 8-12 learners and at most four final-project teams. Bring your own examples and tell the organizer what support would help.

## Schedule and deadlines

Five weekly, 120-minute meetings. Actual dates, time, and location stay unset until an organizer configures a cohort. Assignments 1-3 are due at 6 pm the day before the next class, the charter at Session 4's start, and the final revision 72 hours after Session 5 ends. Class times and other deadlines use the cohort timezone. The final demo-ready candidate is due 24 hours before Session 5. Agree extensions when needed; there are no automatic late penalties.

## Learning outcomes

- Turn a real need into one small journey and observable success checks.
- Explain how a browser, server, API, and database fit together in your app.
- Check sources and combine datasets without hiding missing values, uncertain matches, or limitations.
- Test persistence and access rules, and keep server credentials out of public code.
- Plan with an agent, inspect a change, test it, and explain the decisions you kept for yourself.
- Use branches, reviews, tickets, and handoffs so a teammate can continue without your chat.
- Demonstrate a result, explain a revision, and respond thoughtfully to feedback.

# Tools and setup

- One coding agent: Codex is the reference workflow; Claude Code or another supported agent is fine. Keep approval controls on. Use a course-only folder, not a folder full of personal files.
- Browser tools: Inspect the screen and one network request. Ask for help; terminal commands are not a prerequisite.
- GitHub: Save recoverable checkpoints, branches, and reviewed changes. Private repositories are allowed; everyone uses their own account.
- Supabase / Postgres lab: Learn tables and access policies using fake records in a disposable database. The SQL lab and browser explanation support learners without a cloud project.
- Vercel or a local run: Test a preview URL on another device. Local run instructions and a backup demo are accepted when hosting access is unavailable.
- Collavibe: Use a project team for actual group building. Your course cohort may be a different team; private intake and submissions go to its organizer.
- Confirm laptop, browser, agent access, and a way to run a web project before Session 1. Offer a setup clinic or a working paired environment without sharing passwords.
- Create a course-only folder. Ask the agent what it can read and change, and keep unrelated files outside it.
- Create or join the organizer's course team and complete the private intake. Keep passwords and agent keys out of submissions.
- Prepare fictional test data. No real user emails, health information, payment details, or student records are needed.
- Check captions/transcripts and reading alternatives. Bring a written question if watching a video is not suitable.

# Working agreements

## Using AI

Use the agent for implementation, explanations, and test ideas. You choose the scope, confirm consequential actions, inspect results, and write your reflection. State which help you used. An AI draft does not establish understanding or successful testing.

## Costs and access

Use guided no-key APIs and the agent arrangement confirmed by the organizer. Do not enter a billing card or buy a plan for an assignment. If a limit blocks work, use the labeled offline/local route and ask for support.

### Privacy and security

Use fake data, restrict access, and keep server secrets off the client and out of Git. If a credential is exposed, stop using it, notify the organizer, and revoke/rotate it with the account owner. Deleting one screenshot does not revoke a key.

### Participation and absence

Use a spoken, written, diagram, or demo role to explain your thinking. If you miss class, use its activity brief and arrange a short partner/organizer walkthrough. Agree a revised deadline; never fabricate attendance or peer feedback.

### Feedback and completion

This is a formative, ungraded course. The organizer uses each brief's observable criteria and gives a concrete next step: keep revising or ready. Revise and resubmit. There is no automatic AI grading or public ranking.

### Sharing

The showcase can stay within the class. Get agreement before recording, inviting an external audience, publishing a story, or sharing someone else's work. Public portfolios require an explicit preview and publication decision.

# Feedback

- **Keep revising:** An important criterion lacks evidence or the main task fails. Name the missing test/decision and agree a manageable revision.
- **Ready:** The task and required evidence are present, limits are honest, and the learner can explain a key decision. Optional features are not required.
- **Extend if useful:** The core is ready. Improve accessibility or investigate a deeper trade-off without expanding everyone else's workload.

# Session 1: Direct the agent and test a first build

Outcome: A prototype, a clear user brief, and a tested revision.

## Before class

- 10 minutes: How a coding agent works. Watch 31:30-41:50 or use the reading alternative. Name an action you would want the agent to ask about first.
- 15 minutes: Set up Codex. Confirm access and open a course-only folder. Tell the organizer if setup is blocked; do not buy a plan to keep up.
- 5 minutes: Complete intake and bring one everyday app frustration.

## In class: 120 minutes

- **0-10 min: Welcome, ideas, and working agreements.** Ask for a problem each person cares about. Explain private-by-default work, no rankings, and how to ask for help. Output: One user/problem sentence.
- **10-25 min: Model Plan → Build → Check.** Build a tiny checklist, narrate a scope choice, test an empty input, inspect the change, and show a recovery checkpoint. Output: A shared example of evidence.
- **25-35 min: Compare four-sentence briefs.** A partner asks who the user is and how success will be checked. Remove one unnecessary feature. Output: Person, problem, interaction, check.
- **35-65 min: Build one working interaction.** Learners direct their agents. Help someone explain the request before helping them add features. Output: A running prototype.
- **65-75 min: Break.** Pause screens and offer a quiet check-in. Output: A real break.
- **75-95 min: Partner test and revision.** Testers attempt the task without coaching. Observe, then change one thing. Output: An observed problem and a revision.
- **95-110 min: One-minute demonstrations.** Allow twelve one-minute demos plus transitions. Discuss what testing revealed, not whose app looks best. Output: An example and a question to borrow.
- **110-120 min: Exit note and Assignment 1.** Learners explain a choice and uncertainty in their own words. Check that everyone can save and run the work. Output: A checkpoint and feasible home plan.

## Activity

Build for a need from your own life. Keep the first version small enough to test in a minute. The agent may write all the code, but you should know what the main files do and why you chose this interaction.

## Discussion

- What did I decide before the agent built anything?
- What did testing reveal that the first screen did not?

### Exit check

Show the interaction and name a test that changed it.

### Between classes

Finish Assignment 1 (75 minutes). Prepare for Session 2 (30 minutes).

### Facilitator note

If setup takes over, pair learners with a working course environment and arrange a clinic. This is not a terminal installation contest.

## Session 2: Connect two APIs and question the data

Outcome: A two-source prototype with a justified connection and deliberate failure behavior.

### Before class

- 15 minutes: HTTP: requests and responses. Watch the HTTP short or read its transcript. Distinguish a request, response, and error status; no memorization quiz.
- 10 minutes: The browser, server, and database. Trace browser → server/API → response for one click in your prototype.
- 5 minutes: Public APIs directory. Bring a data-source idea or choose the guided pair. Do not sign up for a paid service.

### In class: 120 minutes

- **0-25 min: Show Assignment 1 and a tested revision.** Twelve learners get 90 seconds each; use seven minutes for patterns and questions. Smaller groups have more dialogue. Output: A revision and its evidence.
- **25-40 min: Trace one API call.** Use a fixed location and inspect the Network response. Point to the fields shown on screen, then trigger a failure. Output: A request-to-screen sketch.
- **40-55 min: Check two providers before building.** Inspect docs for fields, terms, authentication, limits, and browser support. A directory listing is not verification. Output: Two API checks and one question.
- **55-65 min: Break.** Pause requests as well as screens. Output: A real break.
- **65-95 min: Build a meaningful combination.** Use the starter or a chosen pair. Align coordinates/time, or explain why results should remain separate. Output: A first two-source display.
- **95-110 min: Test partial failure and interpretation.** Make one provider unavailable. Ask a partner what the display means, then correct a misleading label. Output: A failure behavior and revised label.
- **110-120 min: Debrief and Assignment 2.** Explain why the second dataset helps. Reduce scope when the relationship is unclear. Output: A home plan for five checks.

### Activity

Answer one question better than either dataset alone. In the guided route, compare weather and air conditions at the same place/time. For another pair, test the relationship instead of inventing a join.

### Discussion

- What does each measurement represent?
- What stays useful when one service fails?

### Exit check

Explain the sources, their relationship, and a partial failure.

### Between classes

Finish Assignment 2 (105 minutes). Prepare for Session 3 (25 minutes).

### Facilitator note

Keep labeled fixtures ready. An outage is a reliability problem to investigate, not permission to pretend sample data is live.

## Session 3: Save data, restrict access, and deploy honestly

Outcome: A small data model, access-test evidence, and an accurate persistence/deployment claim.

### Before class

- 10 minutes: What a database stores. Watch the first ten minutes of Querying or read the opening transcript. Bring a saved-items table sketch.
- 8 minutes: Row Level Security. Read the introduction with your agent. Write who should read and change each row.
- 7 minutes: Environment variables. Distinguish a server secret from a publishable identifier. Bring a question, never a real key.

### In class: 120 minutes

- **0-30 min: Demonstrate the two-API apps.** Twelve learners get two minutes each, with six minutes for transitions/discussion. Show a response-to-screen explanation and a failed request. Output: Integration and data reasoning.

- **30-45 min: Model a saved item and access rule.** Contrast browser storage, server storage, authentication, and authorization using fake users. Output: A table and own-row rule.
- **45-60 min: Design before generating SQL.** Pairs choose fields and predict three access results. Ask for SQL after the rule is clear. Output: A model and predicted outcomes.
- **60-70 min: Break.** Offer a slower worked example if needed. Output: A real break.
- **70-95 min: Run the database policy lab.** Use disposable SQL or inspect the browser simulation. Compare own-user, other-user, and logged-out results. Distinguish real Postgres evidence from a simulation. Output: An honest access-test record.
- **95-110 min: Preview deployment and check secrets.** Deploy a safe course project or demonstrate locally. Test the URL on another device. Distinguish preview and production. Output: A tested environment and checkpoint.
- **110-120 min: Form teams and brief the tasks.** Use interests/support needs to form teams. Explain the 30-minute charter and individual reliability exercise. Output: Partners and a bounded problem.

### Activity

The agent can write SQL and deployment code. You remain responsible for the model, access rules, and evidence. A hidden button is not authorization. A local-only result is valid when labeled accurately.

### Discussion

- Who can read this row regardless of the screen?
- What exactly did my persistence test establish?

### Exit check

Predict and explain one allowed and one denied request.

### Between classes

Finish Assignment 3 (90 minutes) and the team charter (30 minutes). Prepare for Session 4 (25 minutes).

### Facilitator note

Use only disposable data or the rolled-back SQL lab. Never disable security in an existing project for a demonstration. The browser simulator does not prove a real database policy works.

# Session 4: Build together without losing context

Outcome: An approved scope, parallel work, a review, and usable handoffs.

## Before class

- 10 minutes: Git and GitHub in a visual editor. Watch/read the GitHub explanation. Find the branch and latest commit in your repository.
- 10 minutes: Practice a branch and pull request. Try the browser Hello World branch/pull-request exercise. Explain where review happens.
- 5 minutes: Open Connect agent in the intended project team. Ask for help if the client cannot connect; never share a teammate's key.

## In class: 120 minutes

- **0-15 min: Reliability demonstrations in pairs.** Each person shows a saved item and one access test in 60-90 seconds. Debrief recurring problems as a class. Output: A corrected misconception.
- **15-30 min: Team pitches and scope checks.** Four two-minute pitches plus seven minutes for feedback. Approve a small scope before parallel building. Output: One journey, three checks, three to five tasks.
- **30-45 min: Model MCP and code review.** Read a handoff, choose work with the human, inspect a diff, and save a factual summary. Show proposed versus assigned tickets. Output: A shared workflow.
- **45-55 min: Break.** Check every member's access; a copied structured handoff is the fallback. Output: A real break.
- **55-95 min: Build through separate sessions.** Split interaction, data, or tests. Rotate explanation and review so one experienced learner does not become everyone's keyboard. Output: Compatible pieces with handoffs.
- **95-110 min: Review and integrate one change.** Read the handoff and inspect test evidence before merging. Resolve one shared assumption explicitly. Output: A reviewed integration or clear unresolved dependency.
- **110-120 min: Journal and candidate plan.** Write personal notes, review agent suggestions, and agree the smallest demo-ready version. Output: A feasible group plan.

## Activity

A teammate should be able to continue from your handoff: change, reason, branch/checkpoint, actual checks, risks, next action. The handoff is not a full transcript or an effort score.

## Discussion

- Which assumption did our separate agents need to share?
- Can my teammate continue without my chat?

## Exit check

A teammate explains and tests another person's change.

### Between classes

Budget about 150 minutes per learner for the final work/package plus 20 minutes of preparation. The demo-ready candidate is due 24 hours before Session 5. Keep optional features out until the journey works.

### Facilitator note

Notice support needs through dialogue and review, not contribution counts. A blocked MCP client can use a copied handoff in the team discussion; record the limitation honestly.

## Session 5: Test the whole app and explain the process

Outcome: A tested demonstration, useful feedback, and a final revision.

### Before class

- 10 minutes: A first accessibility review. Try keyboard, focus, labels, and text resize. Record actual checks, not a compliance claim.
- 10 minutes: Rehearse the five-minute demo, agree everyone's role, and save backup screenshots/video. Confirm audience and recording agreement.

### In class: 120 minutes

- **0-10 min: Triage integration risks.** Read handoffs and choose the failure most likely to block the demo. No new features. Output: A stabilization plan.
- **10-35 min: Integrate and stabilize.** Run the shared journey and fix one important problem. Keep a known working checkpoint. Output: A shared release candidate.
- **35-50 min: Run the acceptance matrix.** Check empty data, a failed dependency, keyboard, narrow layout, and unauthorized access where relevant. Output: Actual results and honest limits.
- **50-60 min: Break and audience setup.** Confirm audience, accessibility arrangements, and consent. No surprise recording. Output: A ready showcase.
- **60-100 min: Four-team showcase.** Each team gets five minutes to demo/explain and three for questions. Use eight minutes for transitions/shared synthesis. Output: Product, decisions, and useful feedback.
- **100-115 min: Choose a revision and reflect.** Choose one response to feedback. Each person drafts their own account of a decision and what they can now explain. Output: A final revision and reflection.
- **115-120 min: Confirm the finish line.** The final package is due 72 hours after this class ends. Check the exact cohort deadline. Offer private, written, and demo alternatives to public sharing. Output: An agreed submission plan.

## Activity

Show the useful result and how you arrived there. A reproducible failure you can explain is better than a claim that everything works. Stories and public portfolios are optional after review.

## Discussion

- What changed because someone used or questioned our app?
- What can I explain now that I could not explain before?

## Exit check

Every learner explains one choice and names the next test.

## Between classes

Finish one reasoned revision, submit the group package, and add each person's reflection by the agreed deadline. Public sharing is a separate consent decision.

## Facilitator note

The standard timetable supports four teams. Arrange a longer/parallel showcase for more teams before advertising the course; never compress access accommodations or questions.

# Assignment briefs

### Assignment 1: Build one useful interaction

Individual · About 75 minutes

Due: 6 pm the day before Session 2, in the cohort timezone.

# Purpose

Learn to direct an agent, test what it actually built, and explain one design decision.

Choose a small frustration from your own life. Build a web prototype for one person doing one useful thing. A study-session planner, a packing checklist, or a simple event finder is enough. Keep the data local or fictional for this first version.

# What to do

1. Write a four-sentence brief: intended user, their problem, one action the app supports, and how you will know it works.
2. Ask your agent for a small plan and the simplest implementation. Choose the scope before it edits files.
3. Build the interaction. Ask the agent to explain which files control the screen and behavior.
4. Test a normal input, an empty input, and a narrow screen. Ask a partner to try the main action without your coaching.
5. Make one change based on the test. Save a Git checkpoint and record what changed.

# Submit

- A working link, or local run instructions plus screenshots/demo if publishing is unavailable.
- The brief and the name of the main files; do not paste the whole chat.
- Three test results with expected versus actual behavior, including the partner's observation.
- A short explanation of one choice you made and one limitation you have not solved.

# Evidence checks

- A specific person, need, and success check are clear.
- The main interaction works in the demonstrated environment.
- Normal, empty, and narrow-screen checks are recorded honestly.

- Feedback led to a visible change or a reasoned decision to keep the design.
- You can explain one important choice without reading an AI-written answer.

# Boundaries

- One screen or one short flow is enough. No login, payment, chatbot, or external database is required.
- Do not collect real classmates' personal information. Use fictional records.
- A polished template with no working interaction does not meet the task. Visual complexity earns no extra credit.

# Presentation

Bring a 90-second demonstration to Session 2: 30 seconds for the person/problem, 40 for the interaction, and 20 for the test that changed it.

# Optional extension

After the required checks work, add keyboard support or a clearly explained alternative view.

# Resources

- How a coding agent works - Watch 31:30-41:50 for an agent demonstration. The interface is Copilot; our workflow also works with other coding agents. You do not need a Copilot subscription to watch.
- The browser, server, and database - Read the client/server explanation and request-response examples. Sketch the route one click in your app takes. Skip framework-specific details on a first pass.
- Practice a branch and pull request - Complete the Hello World exercise through a pull request. It works in the browser and does not require programming or a terminal.

### Assignment 2: Make two APIs useful together

Individual, with a peer tester · About 105 minutes

Due: 6 pm the day before Session 3, in the cohort timezone.

# Purpose

Understand how data is requested, combined, interpreted, and shown to a person.

Build an app in which two real APIs contribute to one coherent user task. Browse the public-apis GitHub directory for ideas, then check each provider's own documentation. Two unrelated panels do not count as a meaningful combination. Two independent datasets from the same provider are allowed; two requests to the same dataset, or adding geocoding alone, are not the aim.

# What to do

1. Choose a user question and explain why answering it needs both datasets. Start with a guided pair below if you are new.
2. Complete an API check for each provider: endpoint, useful fields, authentication, current free-use conditions, rate limits, attribution, CORS, and a sample response.
3. Use fixed sample data to sketch the interface first. Keep the sample visibly labeled until real requests work.
4. Fetch a small result from each API. Join on a justified shared key such as location/time; otherwise present distinct results and explain their relationship. Do not join records only because the first result looks similar.
5. Show where data came from, units, relevant time, and any missing coverage. Match coordinates/timezones before comparing weather and air data.
6. Test normal results, an empty search, a missing field, one unavailable provider, and rapid repeated clicks. Keep the working part usable when one service fails.
7. Ask a partner to explain what the data means. Revise one misleading label, assumption, or display.

# Submit

- Working app or an explicitly labeled offline demonstration, repository/run instructions, and a two-minute demo.
- A short API check for both services, with links to primary documentation and the date checked.
- A request-to-screen diagram and a note explaining how the datasets relate.
- Five test results, the partner's interpretation, and one change you made.

# Evidence checks

- Both datasets help with the same meaningful user task.
- The app makes two documented API requests or clearly reports a provider outage.
- Sources, units, time, coverage, and joining assumptions are explained.
- Empty, partial, missing-field, and failed responses have deliberate behavior.
- Requests are bounded; provider terms, attribution, secrets, and privacy are handled.

# Boundaries

- Use read-only public data. No paid API, billing card, scraping, private-data upload, or provider signup is required for the guided option.
- Use one explicit search/refresh action, bounded results, caching where appropriate, and a timeout. Do not poll continuously or fire requests on every keystroke.
- If a provider blocks the browser, use your app's server with a fixed upstream allowlist. Never build an unrestricted URL proxy or disable browser security.
- Data is not automatically accurate or complete. Do not turn this exercise into medical, financial, or other high-stakes advice.
- An offline fixture is a fallback, not evidence that a live API succeeded. Label its source and capture time and tell the instructor what failed.

# Presentation

Session 3: two minutes per learner. Show the combined user task, trace one response into the screen, and demonstrate one failure case. No slide deck required.

# Optional extension

Add a second location or an accessible comparison chart only after the two-source failure cases work.

# Resources

- Public APIs directory - Browse by interest. A directory listing is not a guarantee of free access, current availability, browser support, or permission to reuse data. Check the provider itself before choosing.
- HTTP: requests and responses - Watch the HTTP short, or read its transcript. Focus on requests, responses, and status codes; memorizing protocol details is unnecessary.
- Inspect one network request - Open the Network panel, trigger one search, and inspect its URL, status, and response. Stop there; the advanced performance material is optional.
- Weather API documentation - Use the forecast endpoint for the guided assignment. Check units, timezone, timestamps, attribution, and current non-commercial usage limits. No key is needed for this course's non-commercial example.
- Air Quality API documentation - A separate API/dataset for the same location and time. Identify the air-quality measure and its units. It is modeled data, not a personal sensor reading or medical recommendation.
- Book metadata search - Search works and editions with a small result limit. Missing authors or covers are normal. A book title alone is not a reliable unique identifier.

- Project Gutenberg metadata API - Use a small search result to complement Open Library. Keep separate result lists unless a match can be justified. Copyright availability varies by jurisdiction; do not promise every work is free everywhere.

### Assignment 3: Save data and prove the access rules

Individual · About 90 minutes

Due: 6 pm the day before Session 4, in the cohort timezone.

# Purpose

Distinguish a working screen from reliable storage and correctly restricted access.

Add a small saved-items feature to an earlier app or use the supplied database lab. Design one table and explain who should read and change each row. Use fake users and fake data. Your agent can write SQL; you must be able to explain and test the rule.

# What to do

1. Draw the table with its identifier, owner, content, and creation time. State the read/write rules in plain language before asking for SQL.
2. Run the course SQL lab in a disposable Postgres/Supabase environment, or inspect the browser simulation when cloud access is unavailable. A browser-local saved-items feature plus the real SQL policy lab is a valid core route; wiring a cloud-authenticated app is optional. Do not experiment on an existing personal or production database.
3. With the agent, save an item, refresh/reopen the app, and check that the storage behavior matches the claim. Browser-only storage is not evidence of cross-device persistence.
4. Use the lab's separate users to test own-row access, another user's row, and a logged-out request. Check the database result, not only hidden UI buttons.
5. Inspect the Git diff and deployment configuration for server secrets. Demonstrate a preview deployment or document a local-only result honestly.
6. Before Session 4, discuss possible final-project partners and bring one small user problem to the group-planning activity.

# Submit

- A table sketch and plain-language access rules.
- Evidence of save/read behavior and three access tests, including a denied request.
- A link/run guide, a safe Git checkpoint, and a note explaining which environment you tested.

- One unresolved risk and a proposed next test.

# Evidence checks

- The small data model supports the intended saved-items feature.
- The demonstrated persistence claim is accurate.
- Own-user, other-user, and logged-out access are tested.
- No server secrets are present in browser code, commits, or submitted evidence.
- Cloud, local, and simulated evidence are distinguished honestly.

# Boundaries

- The guided SQL lab is sufficient; building a complete authentication interface is optional.
- Keep Row Level Security on. Never solve a permissions problem by putting a service-role key in client code.
- If you use the browser simulation, it teaches the model but does not establish that a real database policy works. Include the SQL lab output when claiming actual policy enforcement.
- No real personal data, password collection, subscription purchase, or payment processing.

# Presentation

Session 4 opening: show a saved item and one denied access case in 60-90 seconds, or explain a failed test and how you would investigate it.

# Optional extension

Add a timestamped edit or deletion path, with the same access rules and tests.

# Resources

- What a database stores - Watch only the first ten minutes of Querying, or read the opening transcript. Focus on tables, rows, columns, and questions a database can answer. The rest of the lecture is optional.
- Supabase database overview - Find tables, data, and the security guidance. Treat the dashboard as a way to inspect your data model, not proof that access is restricted correctly.
- Row Level Security - Read the introductory explanation and own-row policy examples with your agent. Policies belong in the database, not just in hidden buttons. Never expose service-role or secret keys in a

browser.

- Environment variables - Keep server credentials outside Git and browser code. Environment changes require a new deployment. A publishable client identifier and a server secret have different permissions.
- Deploying to Vercel - Understand preview versus production, then test the URL on another device. Check account eligibility and current limits; the course does not require buying a plan.

### Assignment 4: Agree on a small group project

One plan per group; each member keeps their own reasoning note · About 30 minutes

Due: At Session 4's start.

# Purpose

Start with a shared problem and a feasible scope before splitting implementation across agents.

Form a team of two or three. Talk to two willing classmates about a problem, choose one useful user journey, and agree how you will collaborate. Keep the course cohort separate from your project team if the memberships differ.

# What to do

1. Ask two people about their current workaround without pitching your app first. Ask permission to summarize; do not record identifying details.
2. Write the intended user, one problem, one core journey, and three observable success checks.
3. Choose three to five tasks, each with an owner and a completion check. Record dependencies so people do not unknowingly edit the same foundation.
4. Agree repository access, branches, review/merge responsibility, response expectations, and what happens if someone is unavailable.
5. Choose one project team in Collavibe, connect the actual repository, and confirm each person can access it through their own account and agent.
6. Name one thing you are deliberately leaving out. Bring the plan to Session 4 for a short feasibility review.

# Submit

- A one-page charter using the course template, repository link, and member display names.
- Two anonymized interview observations and the design decision they informed.
- Three to five proposed tasks with completion checks, plus an access/setup check.

# Evidence checks

- The plan responds to an observed need rather than an untested feature list.
- One journey and three success checks fit the remaining time.
- Task ownership, dependencies, and merge responsibility are explicit.
- Every member has a meaningful role and a support/absence plan.

# Boundaries

- Maximum four presentation teams in the standard class plan. With 8-12 learners, use teams of two or three and adjust group sizes accordingly.
- An app for a real person does not require collecting real sensitive data. Use fictional records for the course.
- The organizer must approve a reduced scope when a plan cannot fit the available work time. A smaller complete journey is preferable to a large unfinished concept.

# Presentation

Session 4: a two-minute team pitch, one question from peers, and a facilitator scope check before parallel building.

# Optional extension

Add a sketch of an alternative design and explain why the group chose the current one.

# Resources

- Git and GitHub in a visual editor - Use the official article and linked video. Concentrate on repositories, commits, branches, and publishing changes. The MCP extension portion is optional; your agent can run the Git commands.
- Practice a branch and pull request - Complete the Hello World exercise through a pull request. It works in the browser and does not require programming or a terminal.
- What an MCP connection adds - Optional excerpt 47:00-53:50. Compare the demonstrated MCP with Collavibe: tools expose specific capabilities, not automatic access to a whole conversation.

### Assignment 5: Build, test, and explain a group app

Teams of two or three; individual reflection · About 150 minutes

Due: 72 hours after Session 5 ends; demo-ready candidate 24 hours before Session 5 starts.

# Purpose

Demonstrate a small working product and a collaborative process that another person can understand.

Build the single journey in your approved charter. The final app must use at least one external API or a persistent data store; both are optional. It does not have to contain an AI-powered feature. The important work is directing the agent, testing the result, making decisions together, and handing off context.

# What to do

1. Before Session 5, finish a narrow release candidate and submit a draft/demo-ready version at least 24 hours before class. Split the work fairly; each member should budget about 2-2.5 hours, not 2.5 hours of mandatory synchronized meetings.
2. Use separate coding-agent sessions. Each person begins by reading relevant context and ends by saving a factual handoff. Write your own short journal note; do not ask the agent to invent what you learned.
3. Review proposed tickets before treating them as assigned work. Inspect a branch or diff and run the relevant checks before merging.
4. Ask someone outside your building pair to attempt the main task. Capture the observation and revise or explain why you did not change the design.
5. Run the acceptance matrix: normal journey, empty input/data, failed dependency, logged-out/unauthorized behavior if applicable, keyboard, and a phone-width layout. Record actual results.
6. Prepare an eight-minute showcase slot: five-minute demonstration and explanation, then three minutes of questions. Every member contributes; agree an accessible speaking/written role.
7. After showcase feedback, make one final revision or document why you retained the design. Submit the final package within 72 hours after Session 5 ends.

# Submit

- The app URL or reproducible local run instructions, plus a short backup demo or screenshots.
- Repository access or an agreed source snapshot for the organizer. Do not make a private repository public just to submit it.

- The charter, final scope, architecture sketch, dependency/data source notes, and setup instructions without real keys.
- The acceptance matrix, one outsider test, changes after feedback, and known limitations.
- Selected Collavibe handoffs/tickets that explain the sequence of decisions, not a full chat dump or contribution ranking.
- Each member's 150-250-word personal reflection or equivalent 1-2-minute spoken account: one decision, one revision, what you can now explain, and what remains uncertain.

# Evidence checks

- One coherent user journey works in the environment described.
- Testing includes failures and permissions where relevant, with honest limitations.
- The group can explain architecture, data choices, and a consequential decision.
- Handoffs and reviewed work let another person continue; roles are meaningful.
- User and showcase feedback led to a reasoned revision or a documented choice.
- The demo, run guide, and reflection make both product and process understandable.

# Boundaries

- No payment processing, regulated advice, sensitive personal data, custom model training, native mobile release, or large multi-role platform in the core project.
- No mandatory public posting. A private demonstration, written story, or limited-access video can satisfy communication requirements.
- Do not compare effort through commits, line counts, timers, or AI-generated contribution scores. Review work and learning through evidence and conversation.
- A broken live service does not erase learning: show the labeled fallback, the failure evidence, and the next test. Do not present cached data as live.
- Recording or inviting an outside audience requires participants' agreement. The facilitator arranges an internal audience if public sharing is not appropriate.

# Presentation

Session 5 gives four teams eight minutes each, with eight minutes for transitions and shared feedback. Extra teams require a parallel showcase or a longer session; do not silently shorten everyone's participation.

# Optional extension

Only after the required journey passes: add a second tested workflow, improve accessibility, or prepare an explicitly reviewed public portfolio.

# Resources

- A first accessibility review - Try keyboard navigation, visible focus, labels, headings, text resize, and meaningful image alternatives. These checks help find problems; they do not certify accessibility compliance.
- Practice a branch and pull request - Complete the Hello World exercise through a pull request. It works in the browser and does not require programming or a terminal.
- Environment variables - Keep server credentials outside Git and browser code. Environment changes require a new deployment. A publishable client identifier and a server secret have different permissions.
- Deploying to Vercel - Understand preview versus production, then test the URL on another device. Check account eligibility and current limits; the course does not require buying a plan.

# Two-API routes

## Weather + air conditions

Someone comparing conditions in a city before planning their day.

Query two Open-Meteo services for matching coordinates and compatible times. Weather and air quality are independent datasets even though they share a provider.

Scope: Start with fixed Philadelphia coordinates and one current result from each. A city search is optional; geocoding does not replace the second substantive API.

Watch for: A model is not a personal sensor or medical recommendation. Show sources, units, and time; do not label a location universally safe.

Fallback: The starter includes synthetic fixtures and partial-failure controls. They are explicitly not live readings.

## Book discovery + available editions

A reader exploring a title and related public-domain editions.

Search the same title/author in Open Library and Gutendex. Keep result sets distinct unless title, author, and edition evidence justify a match.

Scope: A user-triggered search, a small result limit, and links to provider records. No bulk download or scraping.

Watch for: Similar titles can be different works. Covers are not identifiers and copyright availability differs by jurisdiction.

Fallback: Use a small attributed example response with its capture date, or fictional records marked as teaching data.

### Choose your own pair

A person and question you can describe clearly.

Find two candidates in the directory, then verify both through provider docs. Explain which field, time, or context relates the results.

Scope: Read-only HTTPS requests, small results, appropriate terms, no required payment, and no sensitive data. Show one sample request per service before building.

Watch for: Free, public, no-key, CORS-enabled, and openly licensed are different claims. Verify each instead of trusting the directory or an AI answer.

Fallback: Choose the guided route when access, licensing, browser support, or reliability cannot be established in time.

## Copyable templates

### Four-sentence brief

```
The person is…
They struggle with…
The one action this app supports is…
I will know it works when…
```

### Plan → Build → Check

```
Build [one interaction] for [person]. First explain the smallest plan, files/services you would change, and checks to run. Wait for my choice. Use fictional data and no paid services. After a small change, show what changed and what you actually tested. Ask before publishing, spending money, using credentials, or changing unrelated work.
```

## Two-source API check

User question:
Provider + official docs:
Endpoint and sample query:
Fields, units, timezone/date, missing values:
Authentication, free-use terms, attribution, limits:
Browser CORS or fixed server route:
Relationship to the other dataset:
Timeout, empty data, partial failure:
Date checked:

## Acceptance matrix

Case | Expected | Actual | Evidence / next change
Main journey | … | … | …
Empty input / results | … | … | …
Missing field / failed dependency | … | … | …
Own / other / logged-out access | … | … | …
Keyboard and focus | … | … | …
Phone layout / larger text | … | … | …

## Team charter

People and participation roles:
User/problem and two anonymized observations:
One journey and three success checks:
Out of scope:
Repository/project team access:
Three to five tasks, owners, dependencies, done checks:
Branch/review/merge agreement:
Communication, absence, support:
Data, cost/limits, fallback:
Showcase roles and sharing agreement:

## Useful handoff

What changed and why:
Feature/ticket, branch/commit:
Checks actually run and results:
Risks and shared assumptions:
Ready for review versus only proposed:
Next useful action:
Exclude secrets, full conversations, and unverified claims.

### Short user test

Ask permission. Describe the task, not the solution.
Please try to [goal] and say what you expect.
Observe without coaching.
What happened and what confused the person?
What will change, and why?
Keep an anonymous note. No private details or recording without agreement.

### Five-minute demo

0:00-0:40 Person and problem
0:40-2:40 One working journey
2:40-3:30 Architecture and data
3:30-4:30 A decision, failure, and revision
4:30-5:00 Limitations and next test
Then three minutes of questions. Everyone contributes through an agreed accessible role.

# Glossary

- **Coding agent:** AI that uses tools to inspect or change a project. You choose the direction and verify the result.
- **Frontend / backend:** The browser interface versus server-side behavior and data access.
- **API:** A documented way for software to request data or an action from a service.
- **JSON:** A common text format for structured data. Inspect fields and values, not just the screen.
- **Database:** Structured storage with rules about reading and changing records.
- **Authentication / authorization:** Establishing which user acts versus deciding what they may access.
- **Row Level Security:** Database policies that restrict which rows a caller can read or change.
- **Repository / commit:** Tracked project history and a named checkpoint of changes.
- **Branch / pull request:** A separate line of work and a proposed integration for review.
- **Deployment:** A running version in a specific environment. Preview and production differ.
- **MCP:** A protocol for calling a service's declared tools, not automatic access to every file/chat.
- **Handoff:** A short factual record that lets another person continue.
- **Fixture:** Saved or synthetic data for testing. Label it so it is not mistaken for current live data.

# Facilitator guide

Default: 8-12 adult beginners, one facilitator, and a helper or planned clinic. At most four final teams of two or three fit the standard showcase.

## Preflight

- Confirm real dates/timezone, venue/meeting access, cohort size, and support arrangements before advertising.
- Confirm agent/device/browser/GitHub access and provider/hosting limits without requiring a billing card.
- Test endpoints, offline fixtures, SQL lab, MCP setup, and backup demos on the teaching network.
- Set cohort dates in Your course and inspect every displayed class/deadline. No cohort dates are implied by the planning schedule.
- Prepare blank, working, and deliberately failing examples. Remove credentials and personal information from projected screens.
- Agree any guest audience, recording, or public sharing and arrange a private alternative.

## Responding to difficulties

- **A learner cannot explain the request:** Return to person/problem/action/check. Ask them to predict the result before prompting again.
- **The agent expands the scope:** Stop and restore the smallest agreed plan. Limit changes to one testable interaction.
- **An API fails or is browser-blocked:** Inspect the response and provider docs once, then use labeled fixtures or the guided pair. Do not fake a live success.
- **An access rule blocks the demo:** Check caller and policy with fictional records. Keep security on; never put a service key in the frontend.
- **One person dominates the group:** Rotate explanation, testing, review, and implementation. Offer support instead of ranking counts.
- **Someone misses class or needs another format:** Use text/activity alternatives, a walkthrough, and an agreed deadline. Do not equate attendance with understanding.
- **The plan will not fit:** Keep one journey and three checks. Remove features while retaining tests and collaboration evidence.

## Feedback

After each submission, name one piece of evidence, one decision to discuss, and one next test. Use the brief's criteria. Ask for a short explanation when understanding is unclear; never infer it from an AI score or commit count.

### Design rationale

Worked examples, supported individual builds, and peer testing precede shared responsibility. The API task makes interpretation visible; the policy lab separates security claims from screens. Group work starts after a common vocabulary and small-project workflow exist. Reflections and handoffs support explanation and coordination, not surveillance.

# Source and resource catalog

Resources are linked, not copied or rehosted. Time allocations are course planning budgets, not claims about entire video lengths. Use the assigned excerpt or reading alternative. Provider plans and interfaces change; recheck before teaching.

- How a coding agent works · Microsoft Learn · Watch. Watch 31:30-41:50 for an agent demonstration. The interface is Copilot; our workflow also works with other coding agents. You do not need a Copilot subscription to watch. Checked 2026-11-02.
- Set up Codex · OpenAI · Try. Follow the official setup for your device. For development, choose Codex and a project folder. Use the agent your organizer has arranged; account access and limits depend on the plan. Checked 2026-11-02.
- Claude Code setup alternative · Anthropic · Reference. Alternative for an existing supported Claude Code account. You only need one coding agent, not accounts with every provider. Checked 2026-11-02.
- The browser, server, and database · MDN · Read. Read the client/server explanation and request-response examples. Sketch the route one click in your app takes. Skip framework-specific details on a first pass. Checked 2026-11-02.
- HTTP: requests and responses · Harvard CS50 · Watch. Watch the HTTP short, or read its transcript. Focus on requests, responses, and status codes; memorizing protocol details is unnecessary. Transcript. Checked 2026-11-02.
- Public APIs directory · public-apis contributors · Explore. Browse by interest. A directory listing is not a guarantee of free access, current availability, browser support, or permission to reuse data. Check the provider itself before choosing. Checked 2026-11-02.
- Weather API documentation · Open-Meteo · Reference. Use the forecast endpoint for the guided assignment. Check units, timezone, timestamps, attribution, and current non-commercial usage limits. No key is needed for this course's non-commercial example. Checked 2026-11-02.
- Air Quality API documentation · Open-Meteo · Reference. A separate API/dataset for the same location and time. Identify the air-quality measure and its units. It is modeled data, not a personal sensor reading or medical recommendation. Checked 2026-11-02.
- Open-Meteo access and usage limits · Open-Meteo · Reference. The free service is for non-commercial use and has usage limits. Recheck before a public launch; do not assume the classroom arrangement covers a commercial product. Checked 2026-11-02.

- Book metadata search · Internet Archive / Open Library · Reference. Search works and editions with a small result limit. Missing authors or covers are normal. A book title alone is not a reliable unique identifier. Checked 2026-11-02.
- Open Library API usage guidelines · Internet Archive / Open Library · Reference. Educational discovery tools fit the intended use. Respect the documented rate limit, cache results, avoid bulk harvesting, and identify regular server-side requests as directed by the provider. Checked 2026-11-02.
- Project Gutenberg metadata API · Gutendex · Reference. Use a small search result to complement Open Library. Keep separate result lists unless a match can be justified. Copyright availability varies by jurisdiction; do not promise every work is free everywhere. Checked 2026-11-02.
- Inspect one network request · Chrome for Developers · Try. Open the Network panel, trigger one search, and inspect its URL, status, and response. Stop there; the advanced performance material is optional. Checked 2026-11-02.
- What a database stores · Harvard CS50 SQL · Watch. Watch only the first ten minutes of Querying, or read the opening transcript. Focus on tables, rows, columns, and questions a database can answer. The rest of the lecture is optional. Transcript. Checked 2026-11-02.
- Supabase database overview · Supabase · Read. Find tables, data, and the security guidance. Treat the dashboard as a way to inspect your data model, not proof that access is restricted correctly. Checked 2026-11-02.
- Row Level Security · Supabase · Read. Read the introductory explanation and own-row policy examples with your agent. Policies belong in the database, not just in hidden buttons. Never expose service-role or secret keys in a browser. Checked 2026-11-02.
- Git and GitHub in a visual editor · GitHub · Watch or read. Use the official article and linked video. Concentrate on repositories, commits, branches, and publishing changes. The MCP extension portion is optional; your agent can run the Git commands. Checked 2026-11-02.
- Practice a branch and pull request · GitHub Docs · Try. Complete the Hello World exercise through a pull request. It works in the browser and does not require programming or a terminal. Checked 2026-11-02.
- Deploying to Vercel · Vercel · Reference. Understand preview versus production, then test the URL on another device. Check account eligibility and current limits; the course does not require buying a plan. Checked 2026-11-02.
- Environment variables · Vercel · Read. Keep server credentials outside Git and browser code. Environment changes require a new deployment. A publishable client identifier and a server secret have different permissions. Checked 2026-11-02.
- A first accessibility review · W3C Web Accessibility Initiative · Try. Try keyboard navigation, visible focus, labels, headings, text resize, and meaningful image alternatives. These checks help find problems; they do not certify accessibility compliance. Checked 2026-11-02.
- What an MCP connection adds · Microsoft Learn · Optional watch. Optional excerpt 47:00-53:50. Compare the demonstrated MCP with Collavibe: tools expose specific capabilities, not automatic access to a whole conversation. Checked 2026-11-02.